CLAUDE CODE

FROM SOURCE

Architecture, Design Decisions,

and Transferable Patterns

18 Chapters

A Deep Dive into Production AI Agent Architecture

Table of Contents

Part 1: Foundations **3**

Chapter 1: The Architecture of an AI Agent 4

Chapter 2: Starting Fast - The Bootstrap Pipeline 8

Chapter 3: State - The Two-Tier Architecture 10

Chapter 4: Talking to Claude - The API Layer 12

Part 2: The Core Loop **13**

Chapter 5: The Agent Loop 14

Chapter 6: Tools - From Definition to Execution 16

Chapter 7: Concurrent Tool Execution 18

Part 3: Multi-Agent Orchestration **20**

Chapter 8: Spawning Sub-Agents 21

Chapter 9: Fork Agents and the Prompt Cache 23

Chapter 10: Tasks, Coordination, and Swarms 24

Part 4: Persistence and Intelligence **25**

Chapter 11: Memory - Learning Across Conversations 26

Chapter 12: Extensibility - Skills and Hooks 28

Part 5: The Interface **30**

Chapter 13: The Terminal UI 31

Chapter 14: Input and Interaction 32

Part 6: Connectivity **33**

Chapter 15: MCP - The Universal Tool Protocol 34

Chapter 16: Remote Control and Cloud Execution 36

Part 7: Performance Engineering **37**

Chapter 17: Performance - Every Millisecond Counts 38

Chapter 18: What We Learned 40

Part 1: Foundations

Before the agent can think, the process must exist. This part covers the foundational architecture that makes Claude Code possible: the six key abstractions, the bootstrap pipeline, state management, and the API layer.

Chapter 1: The Architecture of an AI Agent

What You're Looking At

A traditional CLI is a function. It takes arguments, does work, and exits. grep does not decide to also run sed. curl does not open a file and patch it based on what it downloaded. The contract is simple: one command, one action, deterministic output.

An agentic CLI breaks every part of that contract. It takes a natural language prompt, decides what tools to use, executes them in whatever order the situation demands, evaluates the results, and loops until the task is done or the user stops it.

Claude Code is Anthropic's production implementation of this idea: a TypeScript monolith of nearly two thousand files that turns a terminal into a full development environment powered by Claude.

The Six Key Abstractions

Claude Code is built on six core abstractions. Everything else - the 400+ utility files, the forked terminal renderer, the vim emulation, the cost tracker - exists to support these six.

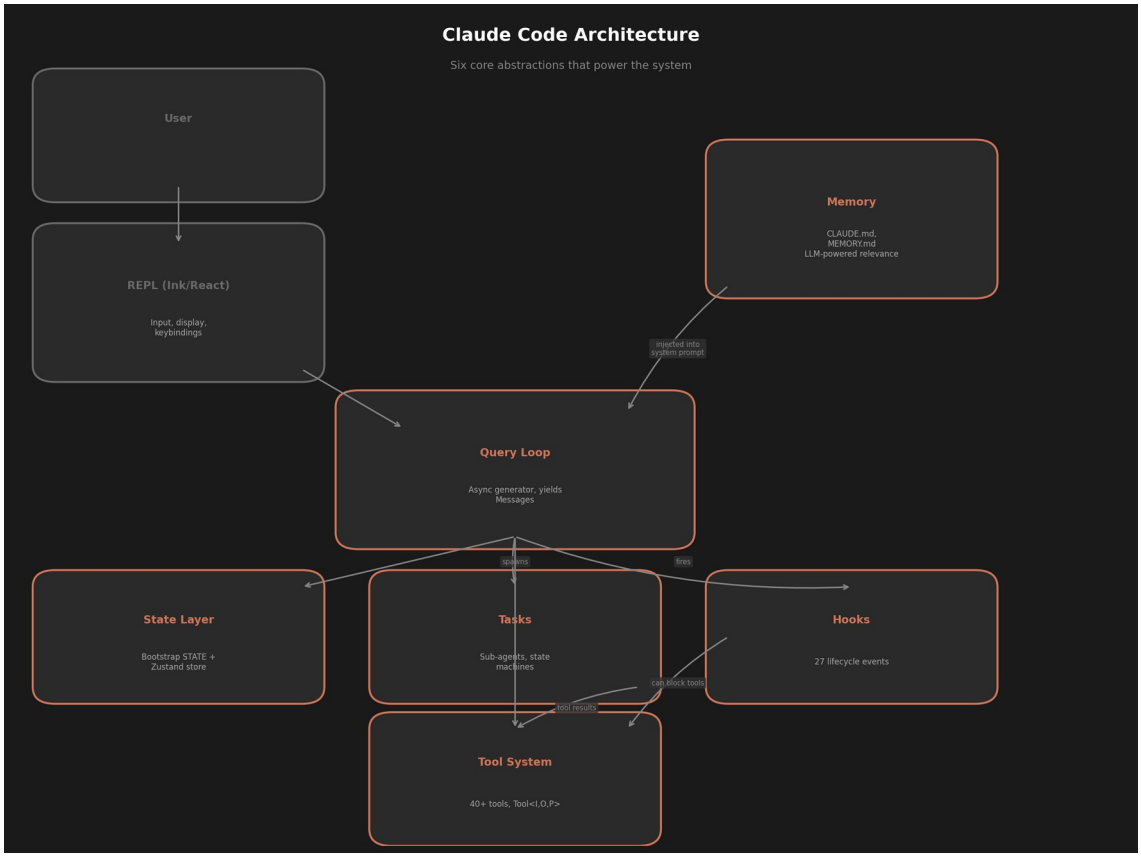

Figure 1.1: The Six Key Abstractions

1. REPL (Ink/React): Handles input, display, and keybindings. The user interface layer.

2. Query Loop: The async generator that yields messages. The beating heart of the system.

3. Tool System: 40+ tools with the Tool<I,O,P> interface. The nervous system.

4. Tasks: Background work units - primarily sub-agents with state machines.

5. State: Two-tier architecture - mutable singleton for infrastructure, reactive store for UI.

6. Memory: Project-level, user-level, and team-level context that persists across sessions.

7. Hooks: User-defined lifecycle interceptors at 27 distinct events.

The Golden Path: From Keystroke to Output

Trace a single request through the system. The user types "add error handling to the login function" and presses Enter.

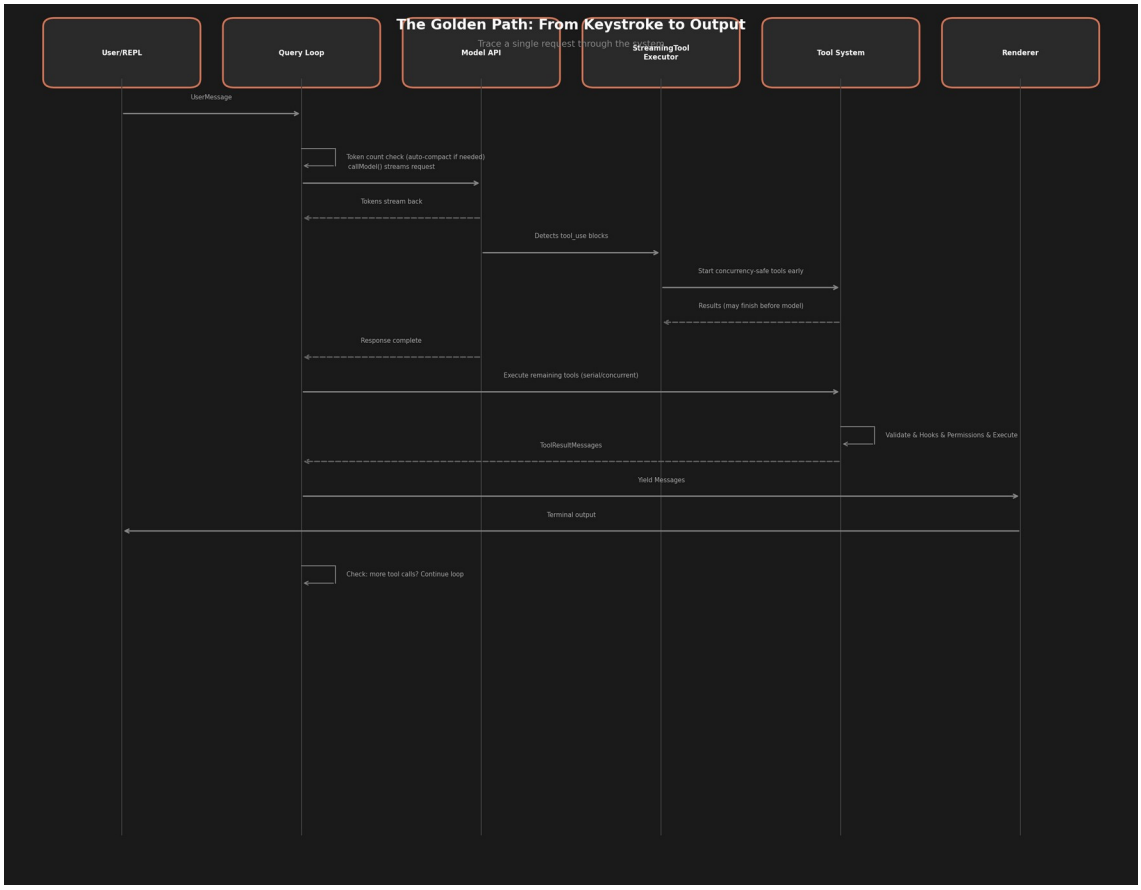

Figure 1.2: The Golden Path - Sequence Diagram

Three things to notice about this flow:

First, the query loop is a generator, not a callback chain. The REPL pulls messages from it via for await, which means backpressure is natural.

Second, tool execution overlaps with model streaming. The StreamingToolExecutor does not wait for the model to finish before starting concurrency-safe tools.

Third, the entire loop is re-entrant. When the model makes tool calls, the results are appended to the message history, and the loop calls the model again.

The Permission System

Claude Code runs arbitrary shell commands on your machine. It edits your files. Without a permission system, this is a security catastrophe.

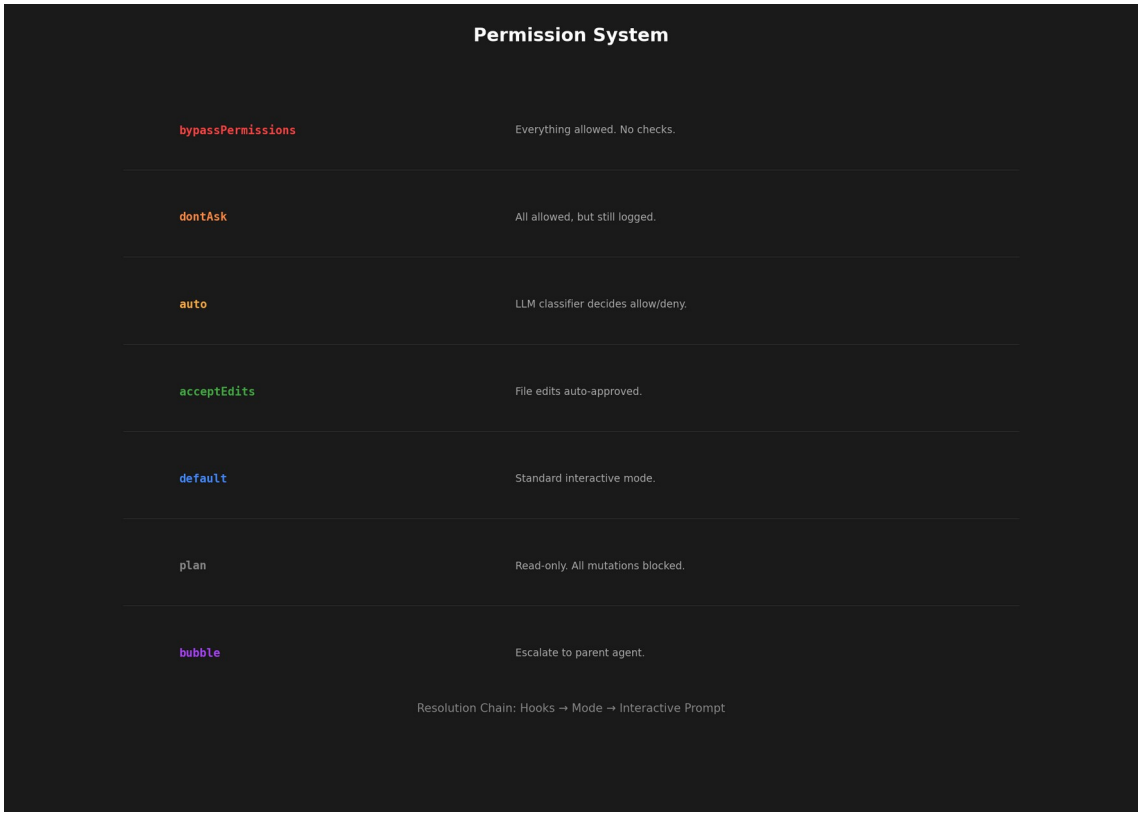

Figure 1.3: Permission Modes

The permission modes are: bypassPermissions (everything allowed), dontAsk (all allowed but logged), auto (LLM classifier decides), acceptEdits (file edits auto-approved), default (standard interactive), plan (read-only), and bubble (escalate to parent).

Apply This

If you are building an agentic system, here are the patterns that transfer:

The generator loop pattern: Use an async generator as your agent loop, not callbacks. Generators give you natural backpressure, clean cancellation, and typed return values.

The self-describing tool interface: Every tool should declare its own concurrency safety and permission requirements. Don't put this logic in a central orchestrator.

Separate infrastructure state from reactive state: Not all state needs to trigger UI updates. Session configuration belongs in a plain mutable object; message history belongs in a reactive store.

Chapter 2: Starting Fast - The Bootstrap Pipeline

The Shape of the Pipeline

The bootstrap pipeline is a five-phase process that takes Claude Code from process start to interactive REPL. Each phase has a specific responsibility and a clean handoff to the next.

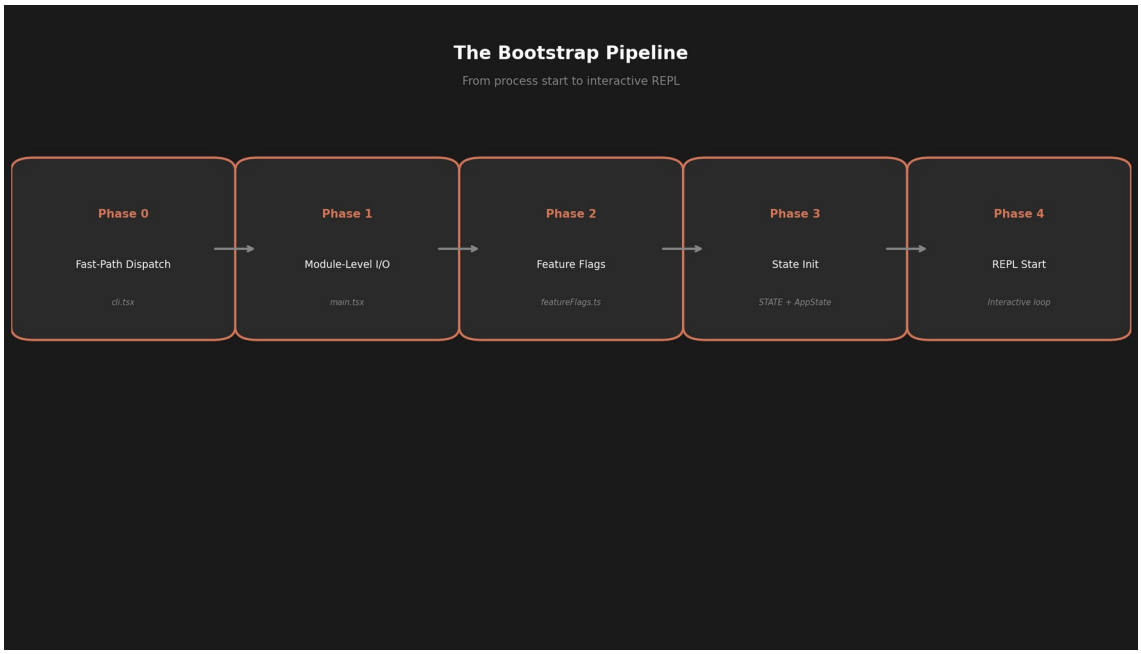

Figure 2.1: The Bootstrap Pipeline

Phase 0: Fast-Path Dispatch

The entry point (cli.tsx) handles fast-path commands like --version and --help without loading the full application. This keeps simple commands instantaneous.

Phase 1: Module-Level I/O

The main.tsx phase sets up stdin/stdout handling, signal handlers, and the basic terminal environment. This is where the process becomes interactive.

Phase 2: Feature Flags

Feature flags are loaded early so that subsequent phases can make conditional decisions. The

system checks environment variables and configuration files.

Phase 3: State Initialization

The two-tier state system is initialized: the mutable STATE singleton for infrastructure, and the reactive AppState for UI.

Phase 4: REPL Start

The interactive loop begins. The system is now ready to accept user input and process queries through the agent loop.

Apply This

Design your bootstrap as a pipeline with clear phase boundaries. Each phase should have a single responsibility and a clean handoff. This makes startup predictable and debuggable.

Chapter 3: State - The Two-Tier Architecture

The Two-Tier Design

Claude Code maintains state at two distinct levels. The separation is intentional: infrastructure state changes rarely and does not need to trigger re-renders, while UI state changes constantly and must.

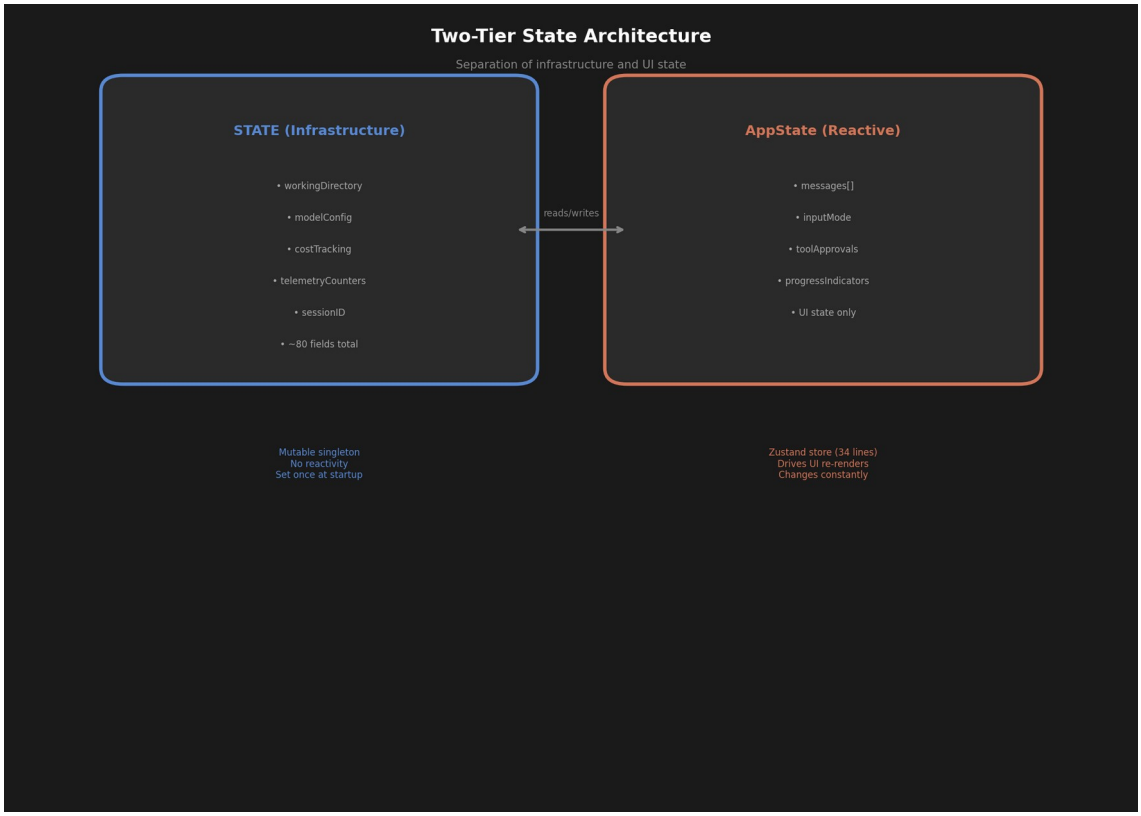

Figure 3.1: Two-Tier State Architecture

STATE: The Infrastructure Singleton

The mutable singleton holds approximately 80 fields of session-level infrastructure: working directory, model configuration, cost tracking, telemetry counters, and session ID. It is set once at startup and mutated directly - no reactivity.

AppState: The Reactive Store

A minimal reactive store (34 lines, Zustand-shaped) drives the UI: messages, input mode, tool approvals, and progress indicators. Changes here trigger re-renders.

Why Two Tiers?

Making everything reactive adds subscription overhead and complexity to state that changes once at startup and is read a thousand times. Two tiers match two access patterns.

Apply This

Separate your state by access pattern, not by domain. Infrastructure state that changes rarely belongs in a plain object. UI state that changes constantly belongs in a reactive store.

Chapter 4: Talking to Claude - The API Layer

The Multi-Provider Architecture

Claude Code talks to Claude through four different infrastructure paths, all transparent to the rest of the system: direct Anthropic API, AWS Bedrock, Google Vertex AI, and local development servers.

System Prompt Construction

The system prompt is assembled from multiple sources: base instructions, memory files, tool descriptions, and context about the current session. This assembly happens fresh for each query.

Streaming

All API responses are streamed. The query loop yields chunks as they arrive, giving the user immediate feedback. Tool calls are detected in the stream and executed speculatively.

Error Handling

API errors are categorized into retryable (network timeouts) and non-retryable (authentication failures). Retryable errors use exponential backoff with jitter.

Apply This

Abstract your API layer behind a common interface. The rest of your system should not know which provider is being used. This enables multi-cloud deployments and local development.

Part 2: The Core Loop

The heart of Claude Code is the agent loop - an async generator that orchestrates the conversation between user, model, and tools. This part explores the loop, tools, and concurrent execution.

Chapter 5: The Agent Loop

The Beating Heart

The agent loop is an async generator, not a callback chain. This is the single most important architectural decision in Claude Code.

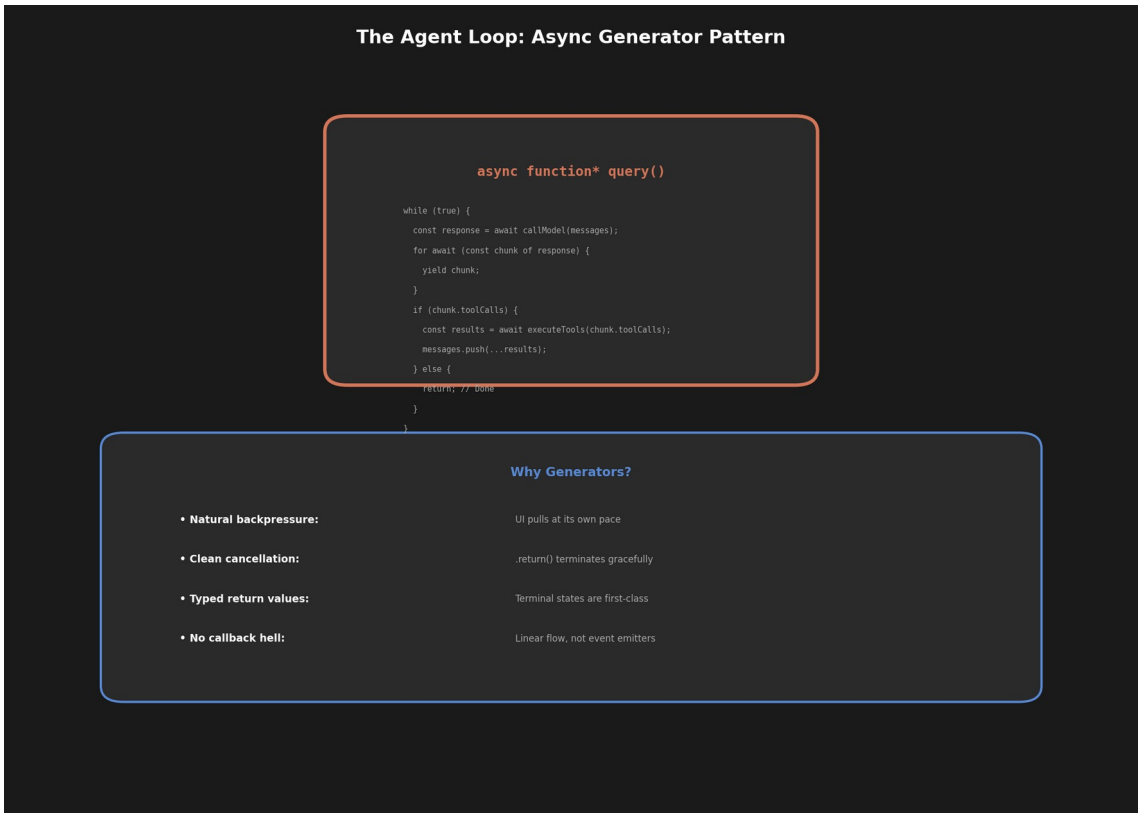

Figure 5.1: The Agent Loop Generator Pattern

Why an Async Generator?

Generators give you natural backpressure - if the UI cannot keep up, the generator pauses. They provide clean cancellation via .return(). And they offer typed return values for terminal states.

The Loop Lifecycle

The loop starts with a user message, calls the model, streams the response, detects tool calls, executes tools, and appends results back to the message history. This continues until the model stops making tool calls.

Termination Conditions

The loop terminates when: the model returns without tool calls, the user aborts, a max turns limit is hit, or the token budget is exceeded. Each termination has a typed return value.

Apply This

Use an async generator for your agent loop. The generator pattern solves problems that callbacks and event emitters cannot: backpressure, cancellation, and typed termination.

Chapter 6: Tools - From Definition to Execution

The Nervous System

Tools are how Claude Code affects the world. 40+ tools provide capabilities from reading files to executing shell commands to spawning sub-agents.

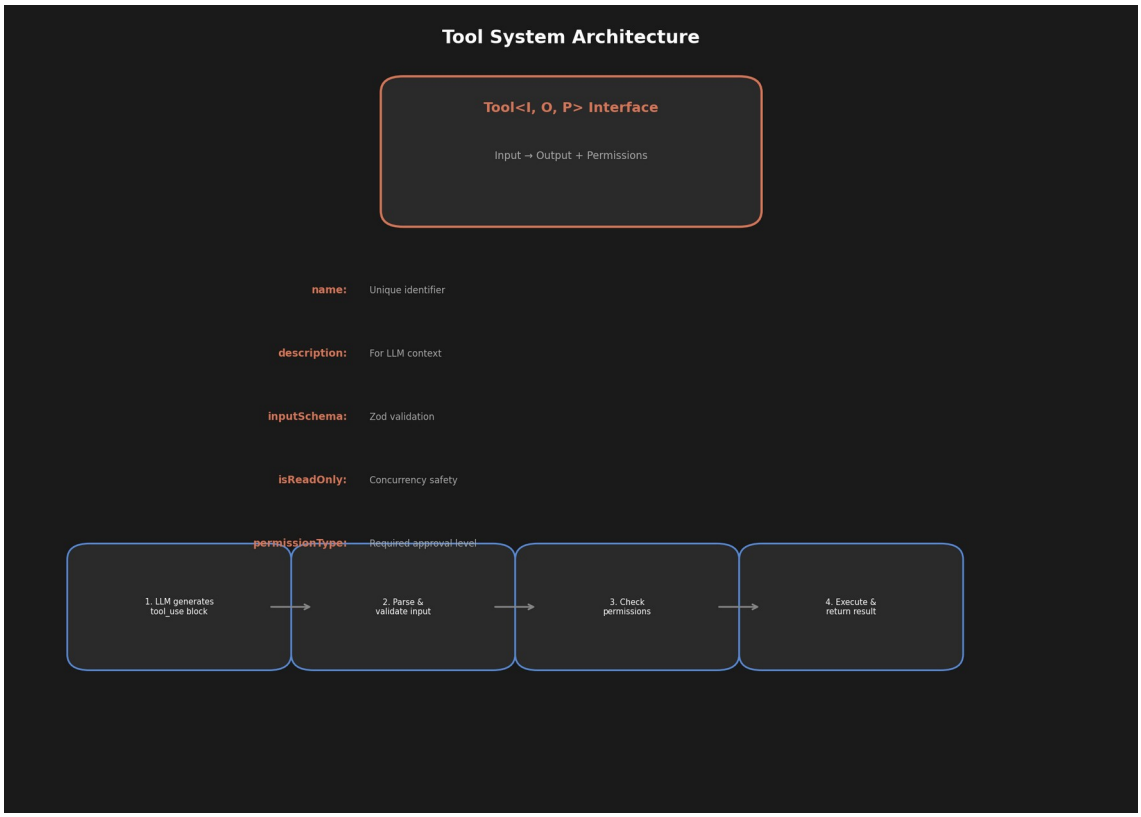

Figure 6.1: Tool System Architecture

The Tool Interface

Every tool implements Tool<I, O, P> - Input type, Output type, and Permission requirements. Tools are self-describing: they declare their own concurrency safety, permission requirements, and rendering behavior.

ToolUseContext: The God Object

The ToolUseContext provides tools with access to state, the API client, and other tools. It's a deliberate god object - tools need broad access to do their jobs.

Execution Flow

Tool execution follows a strict pipeline: parse input, validate with Zod, check permissions, execute, render output, and return results to the loop.

Apply This

Make your tools self-describing. Don't put tool-specific logic in a central orchestrator. Self-describing tools scale linearly - adding tool N+1 requires zero changes to existing code.

Chapter 7: Concurrent Tool Execution

The Cost of Waiting

If tools execute sequentially, the agent waits. A read that takes 50ms blocks a grep that takes 200ms. With 10 tools, you've added seconds to every turn.

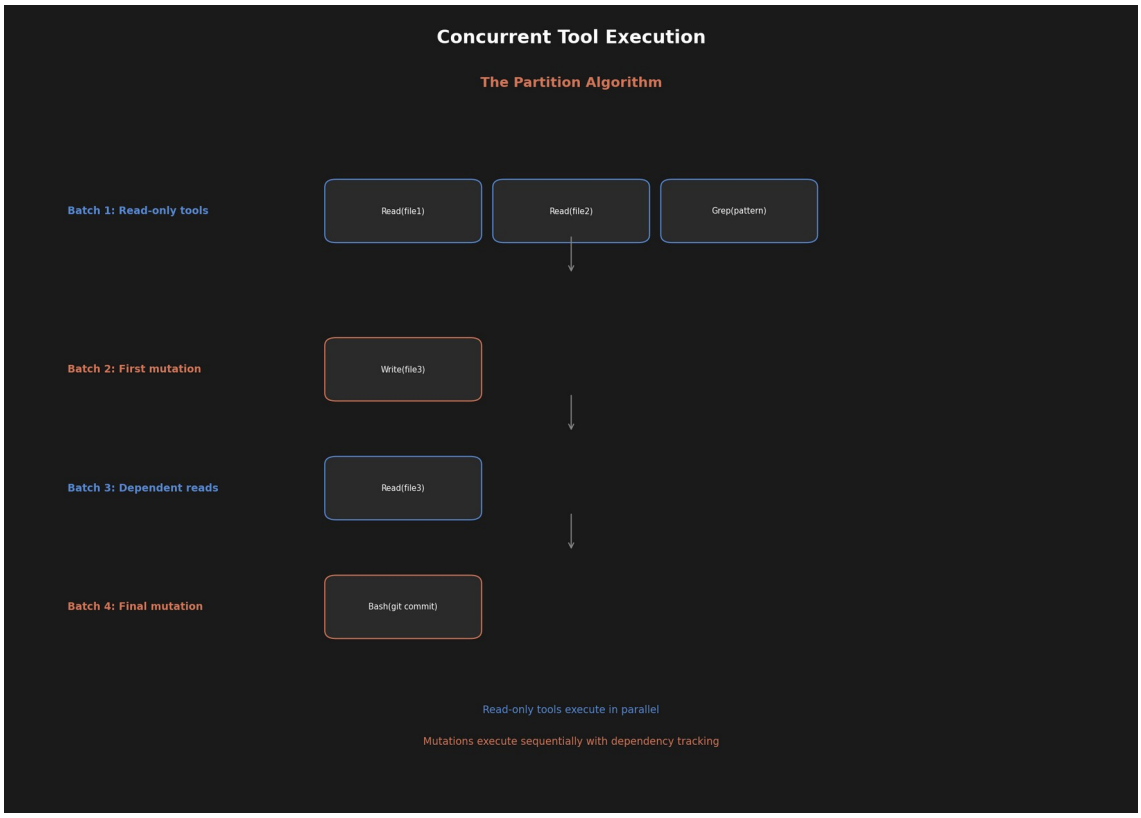

Figure 7.1: Concurrent Tool Execution

The Partition Algorithm

Tools are partitioned into batches. Read-only tools execute in parallel. Mutations execute sequentially with dependency tracking. A tool that reads a file waits for a tool that writes it.

Batch Execution

Each batch runs to completion before the next starts. Results are collected and returned to the loop as a batch. This preserves ordering while maximizing parallelism.

Safety Guarantees

The partition algorithm guarantees that: read-only tools never see partial writes, mutations are ordered by dependency, and the final state is deterministic.

Apply This

Execute read-only tools in parallel. Track dependencies between mutations. Batch results to preserve ordering. This pattern can speed up agent turns by 5-10x.

Part 3: Multi-Agent Orchestration

One agent is powerful. Multiple agents working together are transformative. This part covers sub-agents, fork agents, and task coordination patterns.

Chapter 8: Spawning Sub-Agents

The Multiplication of Intelligence

A single agent can do one thing at a time. Sub-agents multiply that capacity. The parent agent delegates tasks to children, which can delegate further.

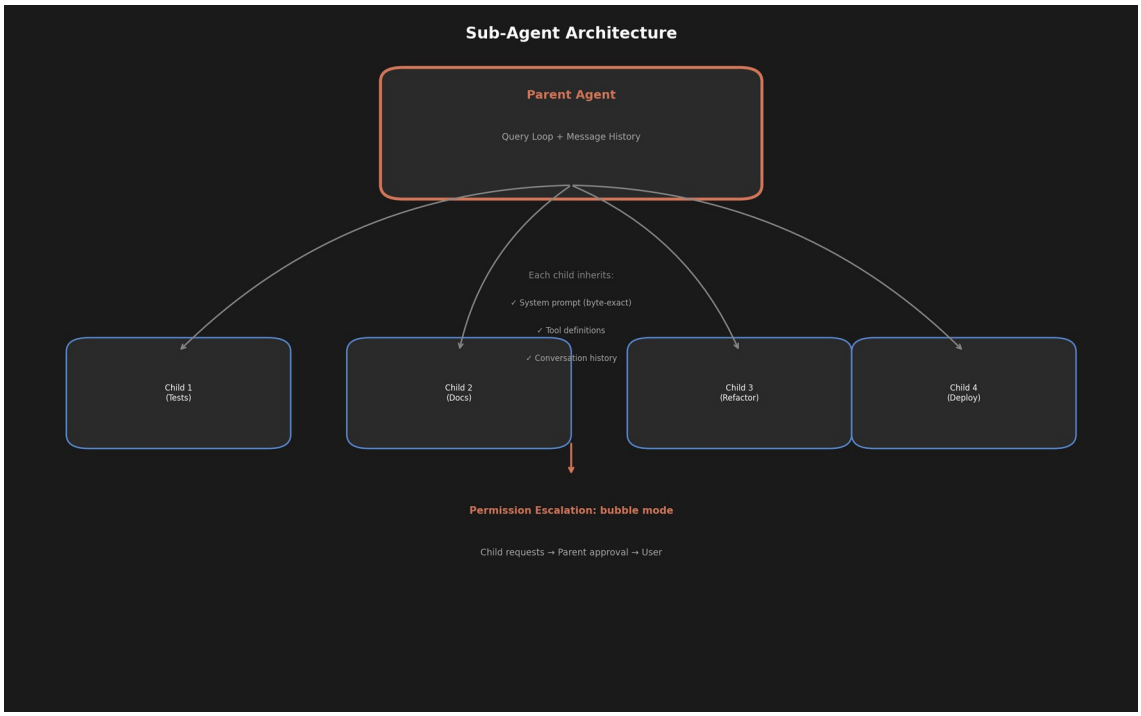

Figure 8.1: Sub-Agent Architecture

The AgentTool Definition

The AgentTool spawns a new query() generator with its own message history, tool set, and permission mode. Children are new instances of the same agent loop.

The runAgent Lifecycle

A sub-agent goes through the same lifecycle as the parent: initialization, query loop execution, and termination. The parent waits for results or lets children run in the background.

Permission Escalation

Sub-agents default to bubble mode. They cannot approve their own dangerous actions.

Permission requests propagate up to the parent or ultimately to the user.

Apply This

Make sub-agents new instances of the same agent loop. Don't special-case sub-agent logic. If the sub-agent is the same loop, it inherits all the same guarantees.

Chapter 9: Fork Agents and the Prompt Cache

The Ninety-Five Percent Insight

When a parent agent spawns five child agents in parallel, the overwhelming majority of each child's API request is identical. The system prompt is the same. The tool definitions are the same. The conversation history is the same.

On a typical fork with a warm conversation, the shared prefix might be 80,000 tokens. The per-child directive might be 200 tokens. That is 99.75% overlap.

The Byte-Identical Constraint

Anthropic's prompt cache gives a 90% discount on cached input tokens. But caching is byte-exact. Not similar enough. Not semantically equivalent. The bytes must match, character for character.

What a Fork Child Inherits

A fork agent inherits four things from its parent by reference or byte-exact copy: the system prompt (not regenerated - threaded), the tool definitions (no filtering or reordering), the conversation history (cloned), and the feature flags.

The Economics

If you can make 80,000 tokens hit the cache for children 2 through 5, you cut the input cost of those four requests by 90%. For the parent, this is the difference between spending $4 and spending $0.50.

Apply This

Design for prompt cache efficiency. Thread system prompts byte-exact. Don't reorder tool definitions. The savings are substantial - 90% off cached tokens.

Chapter 10: Tasks, Coordination, and Swarms

The Limits of a Single Thread

A single agent loop processes one thing at a time. But real work is parallel. While the agent thinks, files could be read, tests could run, documentation could be updated.

The Task State Machine

Tasks follow a strict state machine: pending -> running -> completed | failed | killed. This state machine enables coordination - you can wait for tasks, cancel them, or check their status.

Communication Patterns

Tasks communicate through the parent's message history. A child appends results to the parent's context. The parent can summarize, filter, or ignore child outputs.

Swarm Patterns

Multiple patterns emerge: map-reduce (fan out, collect, summarize), pipeline (stage 1 -> stage 2 -> stage 3), and competitive (multiple agents try, pick best result).

Apply This

Model tasks as state machines. Enable waiting, cancellation, and status checking. Design communication patterns for your use case - map-reduce, pipeline, or competitive.

Part 4: Persistence and Intelligence

An agent that forgets is limited. This part explores how Claude Code remembers across conversations through memory files and extends capabilities through skills and hooks.

Chapter 11: Memory - Learning Across Conversations

The Stateless Problem

By default, LLMs are stateless. Each conversation starts fresh. The model doesn't remember your codebase conventions, your architectural decisions, or your debugging history.

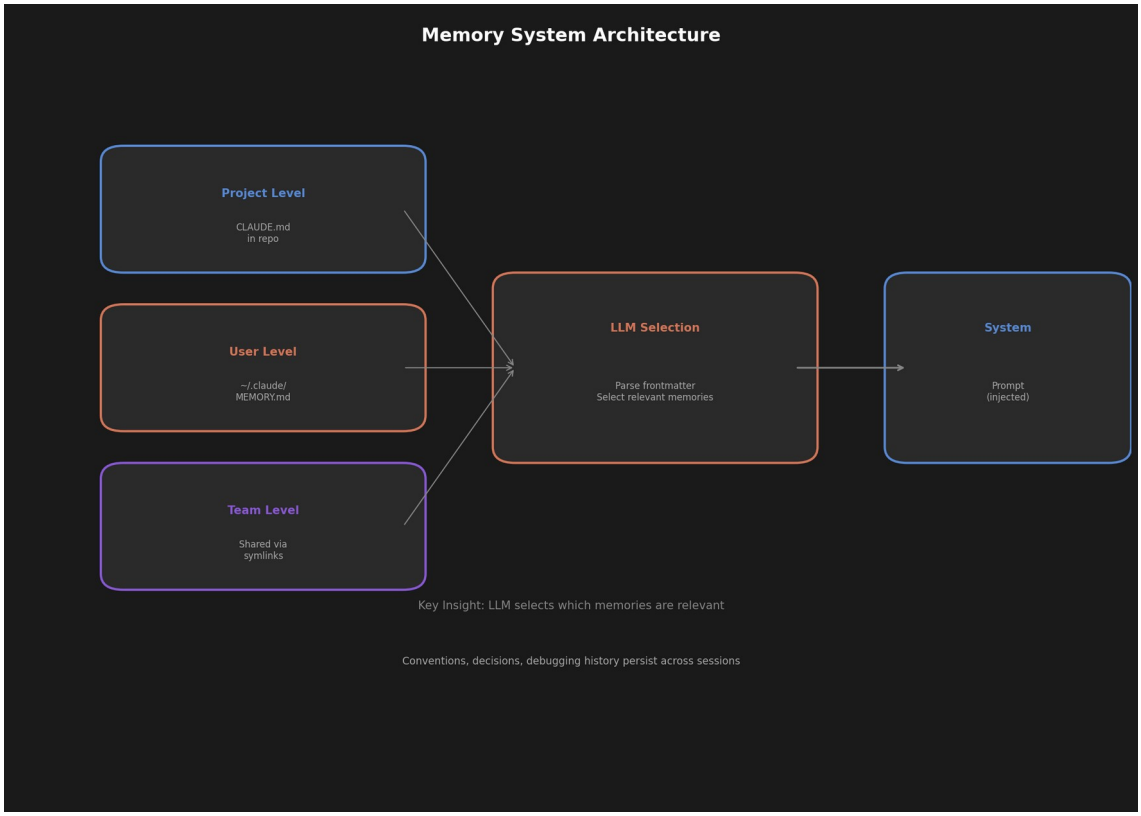

Figure 11.1: Memory System Architecture

The Four-Type Taxonomy

Claude Code uses four types of memory: project-level (CLAUDE.md files in the repo), user-level (~/.claude/MEMORY.md), team-level (shared via symlinks), and session-level (temporary context).

The Write Path

Users write memory files in Markdown with frontmatter. The frontmatter includes tags, scope, and relevance hints. This structured format enables programmatic selection.

The Read Path

At session start, the system scans for all memory files, parses frontmatter, and an LLM selects which memories are relevant to the current conversation. Selected memories are injected into the system prompt.

Apply This

Use an LLM to select relevant memories. Don't dump all memories into the context window. Selection keeps context focused and tokens under control.

Chapter 12: Extensibility - Skills and Hooks

Two Dimensions of Extension

Claude Code extends in two ways: skills teach the model new tricks (custom tools, prompts, and workflows), while hooks control when things happen (lifecycle interceptors).

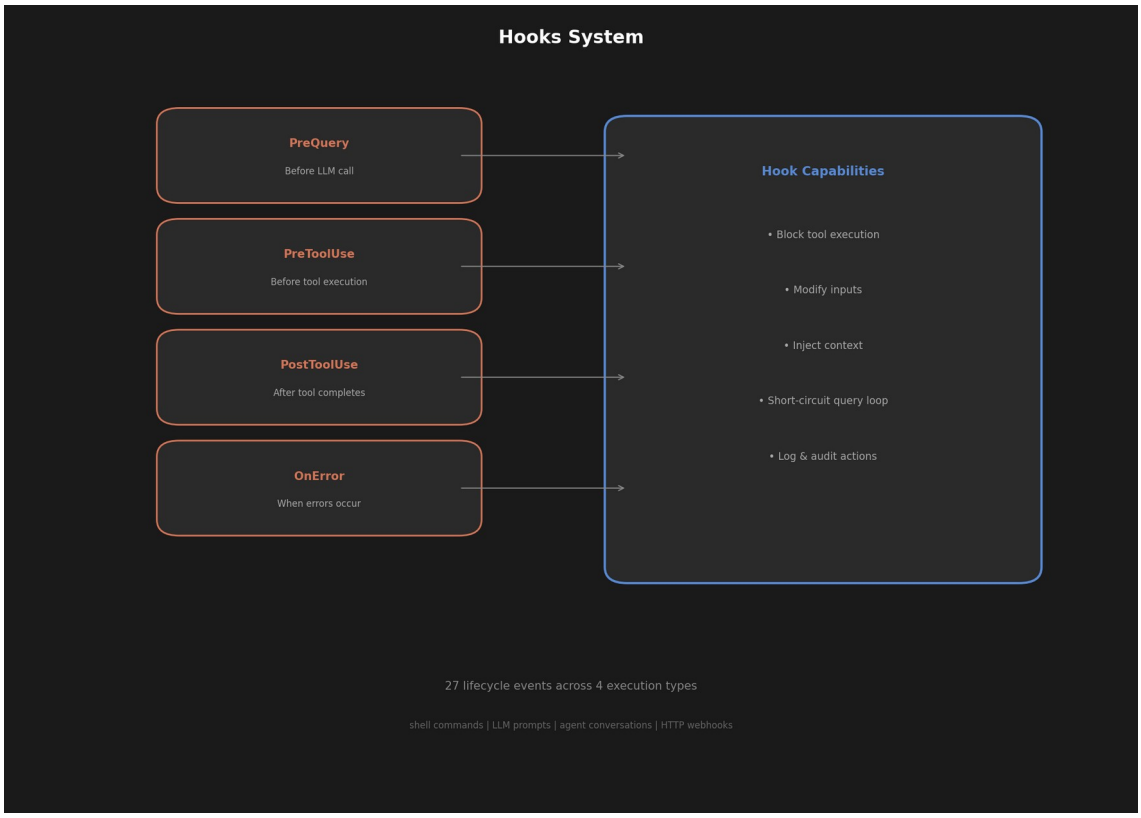

Figure 12.1: Hooks System

Skills: Teaching the Model

A skill is a directory with a manifest and resources. The manifest declares what the skill provides: tools, prompts, and knowledge. Skills are loaded at startup and become part of the system.

Hooks: Controlling Execution

Hooks fire at 27 distinct events across 4 execution types: shell commands, single-shot LLM prompts, multi-turn agent conversations, and HTTP webhooks.

Hook Capabilities

Hooks can block tool execution, modify inputs, inject additional context, or short-circuit the entire query loop. The permission system itself is partially implemented through hooks.

Apply This

Design extension points at two levels: skills for adding capabilities, hooks for controlling execution. This gives users power without requiring them to fork your codebase.

Part 5: The Interface

The terminal is the interface. This part covers the custom terminal UI, input handling, and the rendering system that makes Claude Code feel responsive.

Chapter 13: The Terminal UI

Why Build a Custom Renderer?

Standard terminal output is line-oriented. But Claude Code needs rich output: streaming text, progress indicators, tool status, and interactive approvals. A custom renderer enables this.

The Custom DOM

Claude Code uses a fork of Ink, a React renderer for terminals. The custom DOM provides components for text, boxes, and interactive elements that map to terminal escape codes.

The React Fiber Container

The UI is React. Components render to the custom DOM, which renders to the terminal. This gives Claude Code the full power of React: hooks, context, and component composition.

Rendering Performance

The renderer uses double buffering and dirty-region tracking. Only changed regions are redrawn. This keeps the UI responsive even with rapid updates.

Apply This

If you need rich terminal UI, consider a React-based renderer. The component model is familiar, and the performance optimizations keep things responsive.

Chapter 14: Input and Interaction

Raw Bytes

Terminal input is raw bytes. The escape sequences for arrow keys, function keys, and special characters vary across terminals. A robust input system must handle this variation.

The Key Parsing Pipeline

Input flows through a pipeline: raw bytes -> escape sequence detection -> key identification -> action dispatch. Each stage transforms the input into a higher-level representation.

Multi-Protocol Support

Claude Code supports multiple input protocols: standard terminal escape sequences, Windows console API, and custom key mappings. The abstraction layer hides these differences.

Interactive Elements

The input system supports interactive elements: text input with history, confirmation prompts, and multi-choice selection. Each element handles its own input processing.

Apply This

Abstract input behind a protocol layer. Your application should deal with "arrow up" not "ESC [A". This makes your code portable across terminal types.

Part 6: Connectivity

Claude Code connects to the world through MCP and remote execution. This part covers the universal tool protocol and cloud execution patterns.

Chapter 15: MCP - The Universal Tool Protocol

Why MCP Matters Beyond Claude Code

MCP (Model Context Protocol) is Anthropic's open standard for connecting AI assistants to external tools and data sources. It's not Claude Code-specific - it's a universal protocol.

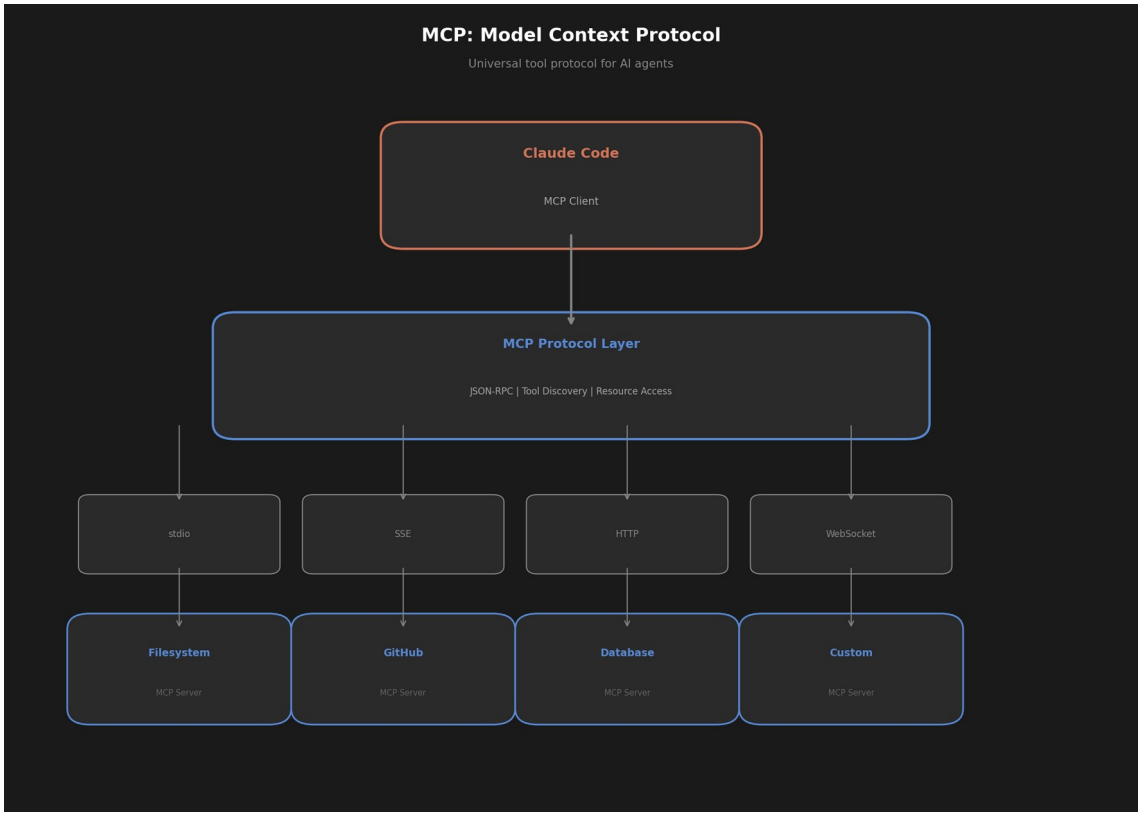

Figure 15.1: MCP Protocol Architecture

Eight Transport Types

MCP supports eight transport types: stdio (for local processes), SSE (Server-Sent Events), HTTP, WebSocket, and more. This flexibility enables connections to any service.

Configuration Loading

MCP servers are configured in claude.config.json. The configuration declares which servers to connect to, their transport settings, and which tools they provide.

Tool Discovery

At startup, Claude Code discovers tools from all configured MCP servers. These tools are added to the tool set and become available to the model.

Apply This

Design your agent to support MCP. The protocol gives you access to a growing ecosystem of tools without writing integrations yourself.

Chapter 16: Remote Control and Cloud Execution

The Agent Reaches Beyond Localhost

Claude Code can run remotely. The agent executes in the cloud while the user interacts through a local terminal. This enables powerful workflows.

Bridge v1: Poll, Dispatch, Spawn

The first bridge protocol uses polling. The local terminal polls for output, dispatches input, and spawns new sessions. It's simple but has latency.

Bridge v2: Direct Sessions and SSE

The second bridge protocol uses Server-Sent Events for real-time streaming. Sessions are established directly, reducing latency and enabling richer interactions.

Security Considerations

Remote execution requires authentication, authorization, and encryption. Sessions are tied to user credentials. All communication is over TLS.

Apply This

Design your agent for remote execution from the start. Abstract the transport layer. This enables cloud deployments and collaborative workflows.

Part 7: Performance Engineering

Every millisecond and token counts. This part covers performance optimizations, from startup time to context window management.

Chapter 17: Performance - Every Millisecond and Token Counts

The Senior Engineer's Playbook

Performance optimization in Claude Code follows the same principles as any senior engineer's playbook: measure first, optimize the hot path, and trade complexity for speed only when justified.

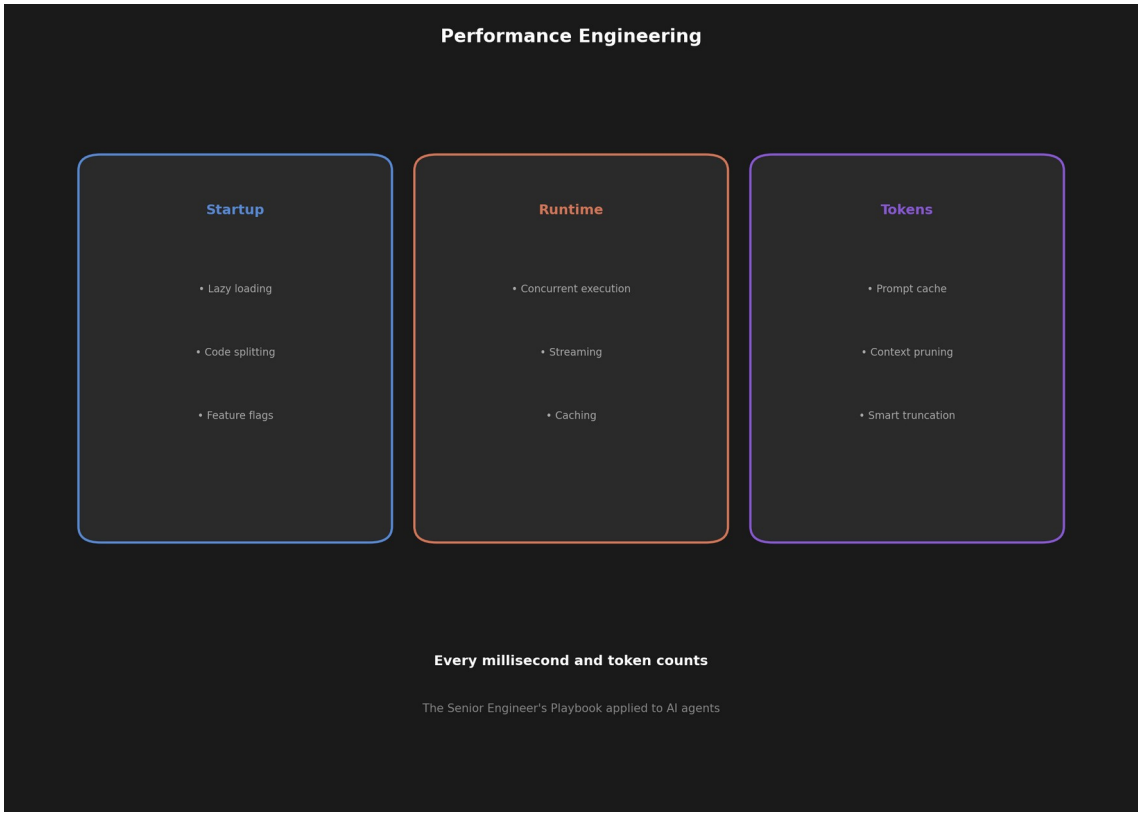

Figure 17.1: Performance Optimization Pillars

Saving Milliseconds at Startup

Startup time is user-perceived latency. Techniques include: lazy loading (don't load until needed), code splitting (load only what you need), and feature flags (skip initialization for disabled features).

Saving Tokens in the Context Window

Context window is money. Techniques include: prompt caching (reuse prefixes), smart truncation (keep recent, drop old), and context pruning (remove irrelevant messages).

Runtime Performance

Runtime speed affects user experience. Techniques include: concurrent execution (parallel tools), streaming (show progress early), and caching (avoid redundant work).

Apply This

Measure before optimizing. Focus on the hot path. For AI agents, tokens are often more expensive than milliseconds - optimize context window usage aggressively.

Chapter 18: What We Learned

Five Architectural Bets

Claude Code made five architectural bets that distinguish it from other agent frameworks:

Bet 1: The Generator Loop Over Callbacks. Most frameworks use callbacks. Claude Code uses an async generator. This gives natural backpressure, clean cancellation, and typed termination.

Bet 2: Self-Describing Tools. Tools declare their own concurrency safety and permission requirements. No central orchestrator becomes a god object.

Bet 3: Two-Tier State. Infrastructure state in a mutable singleton, UI state in a reactive store. Each tier matches its access pattern.

Bet 4: Permission Modes, Not Checks. A small set of named modes (plan, default, auto, bypass) with consistent resolution. Security posture is knowable from the mode.

Bet 5: Recursive Agent Architecture. Sub-agents are new instances of the same loop. Permission escalation flows upward via bubble mode.

What Transfers

These patterns transfer to any agentic system: generator loops for control flow, self-describing interfaces for scalability, tiered state for performance, modes for security, and recursion for composition.

What Does Not

Not everything transfers. Claude Code is a terminal-based tool for developers. If you're building a web-based agent for consumers, your UI layer will differ. If you're building a voice agent, your input system will differ.

The Cost of Complexity

Claude Code is nearly two thousand files. That complexity is justified for a production tool used by hundreds of thousands of developers. Your system may not need that complexity. Start simple. Add complexity only when justified.

Final Thoughts

Architecture is about tradeoffs. The patterns in this book represent tradeoffs made for a specific context: a terminal-based AI coding assistant at scale. Understand the patterns. Understand the tradeoffs. Apply what fits your context.

The future of AI agents is not predetermined. It will be shaped by the systems we build and the patterns we discover. Build thoughtfully. Share what you learn.

Claude Code from Source

Architecture, Design Decisions, and Transferable Patterns